

Appel de fonction

Que se passe-t-il vraiment quand on exécute

```
n = 42
```

```
print("n vaut", n)
```

Appelle la fonction `print`
avec les **arguments** `"n vaut"` et `42`

Définition de fonction

```
def hello_world():
```

en-tête

```
    """Affiche le message 'Hello, world!'  
    """
```

documentation

```
    print("Hello, world!")
```

corps

```
>>> hello_world()  
Hello, world!
```

appel

Fonction avec paramètres

paramètre : une variable

```
def hello(name):  
    """Affiche le message 'Hello, XXX'  
       où XXX est donné par `name`  
    """  
    print("Hello, ", name)
```

argument : une expression

```
>>> hello("Charles")  
Hello, Charles
```

Fonction avec résultat

fonction fructueuse
Ch. 6 – Fruitful functions

```
def hello_message(name):  
    """Retourne le string 'Hello, XXX'  
    où XXX est donné par `name`  
    """
```

```
    message = "Hello," + name
```

```
    return message
```

retourne message
sort de la fonction

```
>>> hello_message("Charles")  
'Hello, Charles'  
>>> msg = hello_message("Charles")  
>>> print(msg)  
Hello, Charles
```

appel = expression

return

```
def test_premier(n):  
    for k in range(2, n):  
        if n%k == 0:  
            print(n, "pas premier")  
            return  
    print(n, "premier")
```

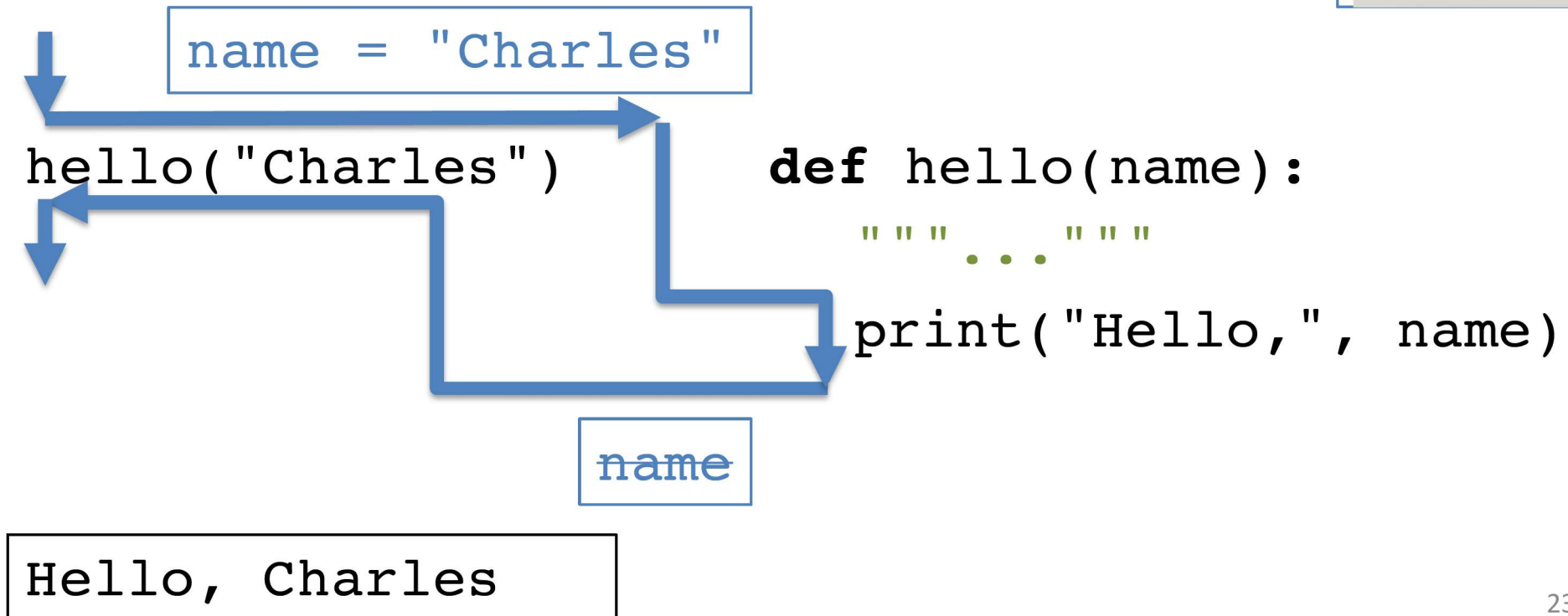
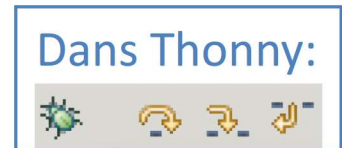


Sort de la fonction

Appel de fonction

Que se passe-t-il vraiment quand on exécute

hello("Charles")



Appel d'une fonction

```
def fonction(paramètres) :  
    " " "documentation" " "  
    corps
```

```
fonction( arguments )
```

- Les **paramètres** prennent la **valeur des arguments**
- Le **corps** de la fonction est exécuté
- L'exécution continue après l'appel

Docstring

```
def hello(name):  
    """Affiche le message 'Hello, XXX'  
       où XXX est donné par `name`  
    """  
    print("Hello,", name)
```

String en tête de définition, documente la fonction

```
>>> help(hello)  
Help on function hello in module __main__:
```

```
hello(name)  
    Affiche le message 'Hello, XXX'  
    où XXX est donné par `name`
```

help(print)

```
>>> help(print)
```

Help on built-in function print in module builtins:

```
print(...)
```

```
print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)
```

Prints the values to a stream, or to sys.stdout by default.

Optional keyword arguments:

file: a file-like object (stream); defaults to the current sys.stdout.

sep: **string inserted between values, default a space.**

end: **string appended after the last value, default a newline.**

flush: whether to forcibly flush the stream.

print()

```
for k in range(10):  
    print(k, end=".")  
print()
```

```
0.1.2.3.4.5.6.7.8.9.
```

```
print(720, 22, 11, sep="-")
```

```
720-22-11
```

Spécification d'une fonction

On veut **décrire rigoureusement** l'effet d'une fonction
= **Spécifier** la fonction

- **Conditions initiales = pré-conditions**
Les conditions sous lesquelles la fonction est applicable
- **Résultat = post-conditions**
Ce que fera ou retournera la fonction exécutée

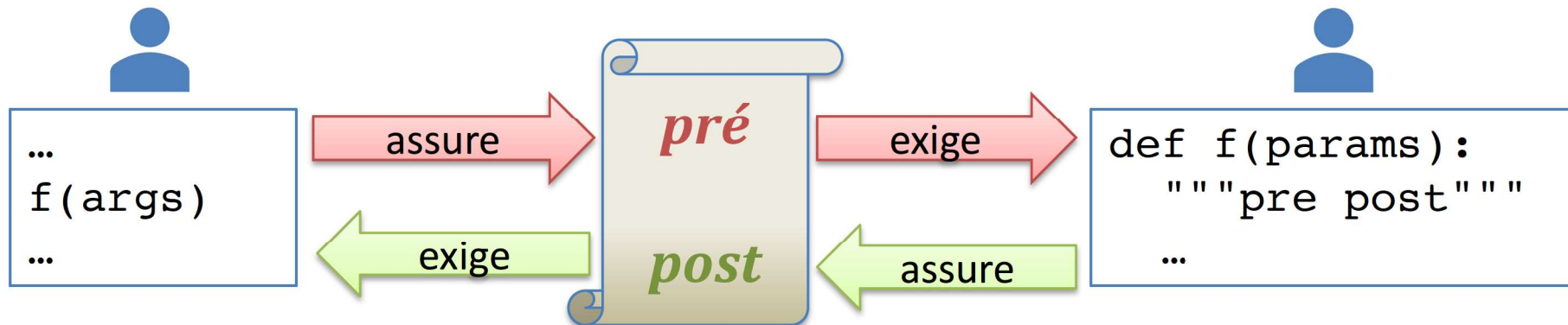
```
def hello(name):  
    """pre: `name` est un string  
        post: Affiche le message 'Hello, XXX'  
        où XXX est donné par `name`  
    """  
    print("Hello, ", name)
```

Contrat d'une fonction

La spécification est un **contrat**
entre la **fonction** et ses **utilisateurs**

Contrat de la fonction

*Si **vos données** satisfont mes **pré-conditions**,
Alors **mes résultats** satisferont mes **post-conditions**.*



Spécifications : exemples

- Racine carrée d'un nombre

```
def racine(n):  
    """pre: n>=0  
        post: retourne la racine carrée de n  
    """
```

- Carré d'un nombre

```
def carre(n):  
    """pre: -  
        post: retourne le carré de n  
    """
```

- Nom d'un mois

```
def mois(n):  
    """pre: n entre 1 et 12  
        post: retourne le nom du n-ième mois  
    """
```

Variables locales

- Les **paramètres** d'une fonction
- Les **variables affectées** dans une fonction sont **locaux** à la fonction
- Il ne sont **pas accessibles en-dehors** de la fonction
- Ils **cessent d'exister** quand la fonction se termine

```
k = 100
r = racine(150)
print(r)
print(k)
print(n)
```

def racine(n):
 """ ... """
 k = 0
 while k*k <= n:
 k += 1
 return k-1

Annotations:

- For `print(r)`: 12
- For `print(k)`: 100
- For `print(n)`: **ERREUR**
- Between `racine(150)` and `def racine(n):`: **n et k sont indéfinis**
- Inside `def racine(n):`: **n et k sont définis et utilisables**